

Article

BIMCoder: A Comprehensive Large Language Model Fusion Framework for Natural Language-Based BIM Information Retrieval

Bingru Liu and Hainan Chen * 

College of Computer Science and Technology, Zhejiang Sci-Tech University, Hangzhou 310018, China;
202230603079@mails.zstu.edu.cn

* Correspondence: hn.chen@live.com

Abstract

Building Information Modeling (BIM) has excellent potential to enhance building operation and maintenance. However, as a standardized data format in the architecture, engineering, and construction (AEC) industry, the retrieval of BIM information generally requires specialized software. Cumbersome software operations prevent its effective application in the actual operation and management of buildings. This paper presents BIMCoder, a model designed to translate natural language queries into structured query statements compatible with professional BIM software (e.g., BIMserver v1.5). It serves as an intermediary component between users and various BIM platforms, facilitating access for users without specialized BIM knowledge. A dedicated BIM information query dataset was constructed, comprising 1680 natural language query and structured BIM query string pairs, categorized into 12 groups. Three classical pre-trained large language models (LLMs) (ERNIE 3.0, Llama-13B, and SQLCoder) were evaluated on this dataset. A fine-tuned model based on SQLCoder was then trained. Subsequently, a fusion model (BIMCoder) integrating ERNIE and SQLCoder was designed. Test results demonstrate that the proposed BIMCoder model achieves an outstanding accurate matching rate of 87.16% and an Execution Accuracy rate of 88.75% for natural language-based BIM information retrieval. This study confirms the feasibility of natural language-based BIM information retrieval and offers a novel solution to reduce the complexity of BIM system interaction.



Academic Editor: Constantinos A. Balaras

Received: 1 May 2025

Revised: 17 June 2025

Accepted: 25 June 2025

Published: 8 July 2025

Citation: Liu, B.; Chen, H. BIMCoder:

A Comprehensive Large Language Models Fusion Framework for Natural Language-Based BIM Information Retrieval. *Appl. Sci.* **2025**, *15*, 7647. <https://doi.org/10.3390/app15147647>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: BIM information retrieval; large language model; natural language based BIM operation

1. Introduction

Building Information Modeling (BIM) is a digital methodology that enables the creation, management, and sharing of comprehensive information about a building or infrastructure project throughout its lifecycle. BIM provides project teams with an efficient collaborative platform by accurately presenting buildings' physical and functional characteristics in digital form. This dramatically improves design accuracy and reduces errors and changes during the construction phase, thus reducing costs and accelerating project timelines. Additionally, BIM offers detailed data support for facility management and operations, which significantly keeps buildings at their optimized performance.

However, BIM information retrieval or model interactions rely on professional software or platforms (e.g., Revit 2024, Navisworks 2024, Bentley OpenBuildings Designer

CONNECT Edition etc.) based on complex structured query language systems [1]. The specialized terminologies and complex operations of existing software systems hinder effective interaction between non-expert users and BIM models. Such challenges directly cause many BIM models to be only used for visual presentation rather than to support building management. Furthermore, even experts often struggle to quickly locate required information from massive BIM data, especially in emergencies where inefficiencies in information retrieval can lead to severe consequences [2]. Therefore, optimizing BIM model interaction and making it more user-friendly is a critical issue that puts BIM models into real construction management work.

Along with the rapid development of natural language processing, building a BIM semantic interaction system based on natural language question-answering is a potential way to reduce the interaction barriers of BIM systems for non-professional users [3]. Such a system does not require users to understand and apply professional programming languages and can respond to users' natural language queries in real time with automatic BIM information retrieval.

In recent years, breakthroughs in large-scale pre-trained language modeling (LLM) have provided powerful tools for computers to understand human natural language. It shows excellent potential in language translation and offers efficient methods for generating structured query statements [4]. Many kinds of LLMs are currently employed in open-domain text generation, e.g., GPT, ERNIE, LLaMA, and Mistral. They exhibit strong language understanding and generation capabilities since they are trained on massive open-domain text data. This study demonstrates that with few-shot prompting or data fine-tuning, these models can seamlessly convert natural language queries into structured computer query language (SQL), the NL2SQL tasks. However, for professional fields such as the construction and building industry, due to limited training data, LLMs inevitably have hallucination problems, which may generate fact errors. Based on the classical SQLCoder model, we trained a BIM-structured query generation model to handle this challenge. To enhance the feasibility of the fine-tuning SQLCoder model, BIMCoder is designed by applying the ERNIE 3.0 model as a Wrapper to preprocess the input of the fine-tuning SQLCoder. By integrating ERNIE and SQLCoder, the BIMCoder model performs well in translating natural language-based BIM information retrieval into structured queries that professional BIM platforms can accept.

The rest of this paper is organized as follows: Section 2 presents the related work. Section 3 illustrates the methodology of the proposed model. Section 4 shows the experiments and results. Section 5 discusses the results, and Section 6 summarizes the conclusions.

2. Related Work

2.1. BIM Models Interaction

As a professional field, the interaction between BIM models and users can generally be grouped into three styles: Graphical User Interfaces (GUIs) [5], vision and hand-motion-based operation (Virtual Reality (VR)/Augmented Reality (AR)) [6], voice-based operation [7].

Graphical User Interfaces (GUIs) remain the standard for interacting with BIM software, providing a familiar but often complex environment. Major BIM platforms like Autodesk Revit [8], Bentley Systems [9], and Dassault Systèmes [10] utilize GUIs that rely on icons, menus, and toolbars to facilitate user interaction with models through mouse and keyboard inputs. For example, In contrast to Revit and MicroStation, which prioritize usability through task-specific groupings and customizable tool arrangements, CATIA is tailored for multidisciplinary engineering applications, offering a highly sophisticated GUI aimed at supporting complex parametric and systems-based modeling workflows.

This makes it more suitable for expert users engaged in large-scale, detailed engineering projects. These systems are primarily developed for the BIM model design stages. While in the BIM model application stages (e.g., buildings construction, operation and maintenance, emergency response and rescue, etc.), the abundance of options in these interfaces can be daunting, particularly for non-professional users, often leading to a steep learning curve [11].

Integrating Virtual Reality (VR) and Augmented Reality (AR) revolutionizes BIM interactions. VR allows users to immerse themselves in a virtual environment, facilitating intuitive exploration and interaction with BIM models. This is particularly advantageous in design reviews and stakeholder presentations, where spatial awareness and real-time feedback are essential [12]. Tools like Unity Reflect and Enscape have advanced VR integration, enabling fully immersive BIM experiences. AR, on the other hand, overlays digital information onto the physical world, allowing users to interact with BIM data within the context of the construction site. Applications like Trimble Connect AR [13] and Dalux BIM Viewer [14] enhance on-site visualization, improving decision-making through a more intuitive understanding of spatial relationships and model details. However, currently, VR/AR-based interaction primarily focuses on optimizing the performance of the vision presentation. The detailed model interaction still relies on menu and icon operation.

Voice-based interaction is emerging as a method to simplify software operations and reduce the need for manual inputs. This approach is instrumental in environments where hands-free operation is beneficial, such as construction sites [15]. Voice commands enable users to interact with BIM models more efficiently, enhancing safety and operational efficiency [16]. H. Zhoui et al. developed a string-matching method to generate navigation queries from each voice navigation request using a fire navigation association dictionary based on Levenshtein distance and Burkhard and Keller (BK) trees [17]. However, challenges remain in ensuring these systems can accurately interpret and execute the diverse commands inherent in BIM operations.

2.2. BIM Information Retrieval

Among all these interaction methods, natural language-based voice interaction is widely recognized as the most instinctive communication method for humans, offering significant user-friendliness and efficiency advantages [18]. Researchers are working to develop models and algorithms that enable systems to understand user queries and extract relevant information from BIM data. These systems typically employ techniques such as text understanding, information retrieval, and question-answering [19]. This approach allows users to express their needs and intentions in everyday language without requiring the learning of specialized command syntax, thereby facilitating efficient communication with BIM systems. This interaction method not only lowers the technical barriers for non-professional users but also significantly enhances the convenience and accuracy of information retrieval, opening broad prospects for promoting and applying BIM technology and the construction industry's digital transformation. Some studies also involve implementing intelligent reasoning and feedback in BIM knowledge question-answering systems [7]. No matter what kind of interaction method, the critical point is retrieving BIM model information. Conventionally, there are three ways: keyword-based retrieval and matching, ontology-based methods, and rule-based methods.

Keyword-based retrieval methods locate and extract relevant entities or attributes within BIM models by identifying key terms within user queries. J.R. Lin et al. proposed the concepts of "keywords" and "constraints" to capture key objects of user interest and designed an improved IFD-based mapping method from keywords to IFC entities or attributes [1]. Z. Hu et al. also proposed a framework for solving intelligent BIM data mining

problems using natural language processing (NLP) and the International Framework for Dictionaries (IFD) [20]. J. Wang et al. utilized natural language processing techniques and IFD technology to parse and unify queries [2]. A. Nabavi et al. proposed a framework that employs a support vector machine (SVM) algorithm to determine the type of user question, utilizes NLP for syntactic analysis, identifies the main keywords of the user's question, and then uses ontology databases such as IFCOWL and NLP methods (latent semantic analysis LSA) for semantic understanding of the problem [21]. Q.S. Xie et al. used IFD to perform semantic disambiguation on key information processed by NLP, thereby obtaining standardized natural language word sequences [22]. Keyword-matching methods can have fast responses. However, they may face ambiguous keyword problems; thus, the performance relies on accurately identifying the keywords.

Ontology-based methods are proposed to handle the keyword's ambiguous issues. S.F. Wu et al. mapped standardized concepts and synonyms obtained from an IFD-based domain ontology through an NLP parser to an IFC-based BIM object database, sorted the results based on a similarity threshold, and obtained the final result [23]. M. Yin et al. developed a modular IFC Natural Language Expression (INLE) ontology that uses ontology-based instance filling and text parsing to extract expected variables and different levels of constraints from NLQ and convert them into standard SPARQL queries that can be used to retrieve IFC-based BIM models [24]. G. Gao et al. proposed an online BIM resource automatic query extension method based on IFC IR ontology and local context analysis (LCA) [25]. Ontology-based methods enhanced the robustness of BIM information retrieval procedures. However, constructing an ontology model relies on experts' experience; thus, building costs are relatively high.

A compromise is a rule-based method. N. Wang et al. used vectorization to locate the possible IFC entity with the highest cosine similarity. Once a possible IFC entity was found, corresponding natural language paragraphs were generated from the IFC entity based on the IFC4 pattern [2]. This approach enables the system to provide higher-level decision support based on BIM data, such as risk assessment, path planning, and building attribute analysis. L.Y. Ding et al. conducted semantic inference based on the particle inference engine called the Protégé API [26]. X.W. Li et al. integrated multi-source security risk factors and text-based supervision into a SPARQL-based inference framework to improve the automation and effectiveness of subway construction safety inspections [27]. Y.H. Zhou et al. developed the DSMS domain ontology (OntoDSMS) by comprehensively collecting domain knowledge and extracting contextual information from the dam information model. They also implemented rule-based reasoning and SPARQL queries [28]. D.M. Guo et al. proposed a simple method for automatically generating SPARQL (SPARQL Protocol and RDF Query Language) queries to achieve effective data extraction [29]. S. Cursi et al. described a semantic bridge platform called S-Enr BIM, which allows BIM to integrate with knowledge-based modeling methods and provides effective expression, management, and sharing of all necessary knowledge in the architectural design process [30]. M. Fahad et al. developed an SBIM reasoner using several preprocessors (IFC to RDF converter, geometry extractor) to construct a semantic library from the input IFC model [31].

In general, keyword-based, ontology-based, and rule-based methods focus on semantic matching between users' input and BIM data. Thus, translating the user input into a structured query is crucial in BIM information retrieval procedures.

2.3. Natural Language to Structured Queries

In natural language processing (NLP), NL2SQL (natural language to SQL) is a natural language processing technology designed to translate natural language queries into

structured query language (SQL). It enables non-technical users to access and interact with relational databases through natural language. In the context of BIM, NL2SQL technology offers a more efficient and user-friendly method for retrieving BIM resources, especially when working with large and complex BIM datasets.

Current research and applications of NL2SQL in the BIM field are developing rapidly. W. Solihin et al. have introduced the integration of spatial operations into standardized SQL queries, thereby enhancing the accessibility of BIM data for various query functions [32]. H. Alzraiee et al. proposed a system for construction project cost estimation using SQL, which facilitates the effective exchange of information with BIM elements through an interoperable information management system [33]. G. Demirdögen et al. developed a medical facility management system based on big data analytics (BDA), BIM, and NoSQL databases, enabling queries and visualization of key performance indicators (FM-KPIs) [34].

However, NL2SQL still faces several challenges, including accurately interpreting complex and diverse natural language expressions, managing incomplete semantic information, and generating precise SQL query statements. To support research in this area, various publicly available datasets, such as WikiSQL, Spider, and CoSQL, have been created. These datasets contain numerous real-world examples of natural language queries and corresponding SQL statements. Current NL2SQL models mainly employ two strategies: they are trained or fine-tuned based on models with sequence-to-sequence (Seq2Seq) architectures [35] or they inherit the ability of pre-trained large language models such as GPT, ERINE [36], Qwen, LLaMA[37], Mistral[38], etc., with prompt engineering [39]. Specially trained or fine-tuned Seq2Seq models can perform exceptionally well on the given dataset, while large language models have substantial flexibility to handle the diversity of natural languages.

Summarizing the development of NL2SQL research, an intuitive way to optimize the performance of transforming natural language to structured BIM query language is to fuse a domain expert model with a pre-trained large language model to obtain good accuracy and robustness simultaneously.

3. Methodology

A fusion strategy is employed to build the BIM structured query string generation model. Expert models can achieve high accuracy, but the generalization ability is generally not strong. Open-domain LLMs can help interpret human nature language points, but there are always “hallucination” problems. There are two main contributions of this work.

- We constructed a dataset of pairs of BIM natural language queries and their corresponding structured query statements, which can be a benchmark for the further study of natural language-based BIM operation.
- We designed and trained a hybrid model (BIMCoder) that generates Building Information Modeling (BIM)-structured query statements.

3.1. BIM Query Dataset

The dataset comprises 1680 samples, categorized into two main types: precise queries and fuzzy queries, to comprehensively assess the model’s parsing capabilities in different contexts. The natural language queries were constructed based on representative usage examples drawn from mainstream BIM software systems. These examples reflect frequently encountered information retrieval needs during real-world BIM operations. The query design was informed by domain knowledge embedded in these application scenarios to ensure practical relevance and authenticity.

The precise query dataset contains 1440 samples, each adhering strictly to IFC standard logic, with clear expression and well-structured format. These samples are primarily used to evaluate the model's accuracy and stability when handling explicit query requirements.

In contrast, the fuzzy query test set includes 240 samples, which simulate natural language inputs from non-expert users. This dataset is primarily used to test the model's robustness and generalization in real-world interaction scenarios. These queries may involve semantic ambiguity, incomplete expression, or colloquial descriptions.

All queries in the dataset are divided into 12 standardized query types, corresponding to common query needs on BIMserver, covering core tasks in BIM data extraction. This design ensures both systematic and representative data, aiding in the training and evaluation of the model's performance across different query categories and providing a high-quality testing benchmark for intelligent query systems in the BIM domain. The precise query dataset is further subdivided into 1200 training samples and 240 test samples, while the fuzzy query test set is exclusively used for validating the model's flexibility in handling ambiguous queries. Initially, all query types were manually created and then augmented using GPT to generate 100 queries per type. The fuzzy query test set specifically aims to evaluate the system's matching capability for fuzzy queries. All the source code and dataset can be obtained from <https://github.com/liubingru66/BIMcoder> (accessed on 4 March 2025).

As shown in Table 1, the dataset structure is as follows.

Table 1. Dataset structure.

Index	BIM Query Type	Natural Language Query Example
T1	Obtain the entire building	Display the entire building
T2	Obtain components with a single type	Show the walls of the building
T3	Obtain components with multiple types	Show walls and concrete slabs within the building, excluding other parts
T4	Obtain a component with a given GUID	Locate the component within the entire system with GUID xxx
T5	Obtain components within a specific space	Define a space: width is 5, height is 5, and depth is 5. Show all the items in this space
T6	Obtain all attributes of the component with a given GUID	List the attributes of the component GUID xxx
T7	Obtain the attributes of the same type of component	List the attributes of all walls
T8	Obtain the attribute using the given component type and attribute name	Show the history of the walls
T9	Obtain components with specific subclass components	Display walls with openings
T10	Obtain component properties with a given GUID	List the floor attributes with floor GUID xxx
T11	Obtain components with given attribute values	Show load-bearing walls
T12	Obtain components using class type value with a given classification system	Show all the components with OmniClass value "21-07 11 13"

To better manage and understand the diversity of BIM queries, the 12 types of IFCQL query instructions can be classified based on the query target object, the query method, and its content. Grouping these queries according to their querying methods allows for clearer

selection of query needs for users and ensures the scalability and flexibility of the query language. The classification reasons are as follows:

1. Object-Based Queries

These queries focus on filtering query targets based on different component types, such as walls, floors, doors, and windows. This category includes T1, T2, T3, and T12. This classification is intuitive and aligns with real-world applications where users often need to perform precise queries for different component types (e.g., walls, floors, doors). For instance, T1 and T2 allow the retrieval of entire buildings and individual component types, respectively, while T3 can handle combined queries for multiple component types. These operations are commonly seen in practical BIM applications.

2. Component Identification or Location-Based Queries

This category of queries focuses on retrieving target components through their unique identifiers (GUIDs) or spatial location ranges. It includes T4, T5, T6 and T10.

Queries based on GUIDs or spatial ranges help users locate specific components or areas. For example, T4 and T5 support locating components via their GUID or spatial range, which is a common query need in architectural design and construction.

Although end users typically do not directly interact with GUIDs in daily operations, GUIDs serve as unique identifiers in the BIM context, ensuring the accuracy and traceability of component queries. In practice, users often start queries based on physical or attribute information of components and later use GUIDs to pinpoint specific components for more refined queries. This method not only enhances query accuracy but also reduces computational overhead from repeated queries, enabling users to efficiently access the required information.

3. Component Attribute or Relationship-Based Queries

These queries focus on filtering based on the attributes of components or their inter-relationships. They are primarily used to extract detailed component information or to query the dependencies between components in the BIM model. This category includes T7, T8, T9, and T11.

Attribute and relationship queries concentrate on retrieving details about the components themselves or the logical relationships between them. These types of queries are theoretically sound because in a BIM model, component attributes (such as material, size, load-bearing capacity) and component relationships (such as containment or relative positioning) are critical elements of the model. By querying component attributes and relationships, users can gain in-depth insights into the components' functionality, performance, and interactions with other components.

Based on the information presented in Table 2, the following examples illustrate Precise Queries and Fuzzy Queries.

Table 2. Examples of Precise Queries and Fuzzy Queries.

Precise Queries	Fuzzy Queries
Locate and retrieve items in the building by setting a space with a width of 5, a height of 5, and a depth of 5	Try using a 5 cubic meter area to see what items can be found
Query the corresponding buildings in the building layout using the GUID: 234	GUID: 234, help me find where the corresponding building section is

In BIM query tasks, precise queries and fuzzy queries represent two different user interaction modes, corresponding to structured professional query requirements and more natural, colloquial expressions, respectively. Precise queries typically have well-defined query targets and clear parameter definitions, such as specifying a spatial range, component

type, or GUID for retrieval. Due to their strong directionality, models usually achieve high accuracy in understanding and executing such tasks.

In contrast, fuzzy queries are closer to the natural language expressions of non-expert users and may include vague descriptions, colloquial phrasing, or context-dependent query needs. For example, users may not directly provide specific values or GUIDs but instead use vague instructions like “see what you can find in this area” or “help me find that wall section.” These queries are semantically more flexible and require the model to possess a stronger ability to understand context and reason, in order to correctly translate fuzzy expressions into executable queries.

In BIM information retrieval, it is crucial to support both precise and fuzzy queries. Precise queries ensure efficiency and reliability in professional environments, while supporting fuzzy queries enhances the user interaction experience for non-expert users, making BIM data access more intuitive and natural.

To ensure the validity and practicality of the constructed dataset, it is essential to verify whether the designed query issues can be correctly executed in a real BIM environment. The verification process serves not only to validate the dataset content but also to assess the practical applicability of the query language and the query framework used. For the 12 query types proposed in the dataset, the verification is carried out by executing the queries within the BIMserver open-source project.

The executable IFC files for real-world scenarios can be accessed in the dataset available on GitHub (tested using BIMserver v1.5.187) .

3.2. Fusion-Based BIM Structured Query Strings Generation

The proposed model, termed **BIMCoder**, is a fusion-based architecture designed to translate natural language BIM queries into executable structured commands. It integrates the domain expertise of fine-tuned models with the flexibility of open-domain language models and adds a verification layer to ensure output quality. The architecture consists of three key components: an *expert model*, a *wrapper model*, and a *validator*.

The **expert model** is fine-tuned using a carefully constructed BIM query dataset to capture domain-specific syntactic and semantic patterns. The **wrapper model**, built upon a general-purpose large language model (LLM), serves as a preprocessing module that standardizes user inputs and enhances compatibility with the expert model. Finally, the **validator** ensures that the output query strings conform to IFC-based JSON syntax and can be successfully executed within BIM platforms.

The overall workflow is illustrated in Figure 1.

The detailed process is as follows:

- **Input:** The system receives user-provided natural language queries targeting BIM-related information.
- **Wrapper Model:** A large language model (LLM) reformulates the input queries into standardized, disambiguated forms suitable for domain-specific parsing.
- **Expert Model:** The expert model, trained on BIM query data, converts the preprocessed input into structured BIM query strings in JSON format.
- **Validator:** The validator checks the syntax and executability of the generated queries to ensure compatibility with downstream BIM platforms.
- **Output:** Validated structured query strings, ready to be executed in BIM systems (e.g., IFCQL-compliant JSON), are returned to the user or interfaced applications.

This fusion-based design improves the robustness, interpretability, and domain adaptability of BIM query generation. By leveraging general-purpose LLMs for preprocessing and domain-specific models for generation, BIMCoder effectively bridges the gap between natural language flexibility and structured query precision.

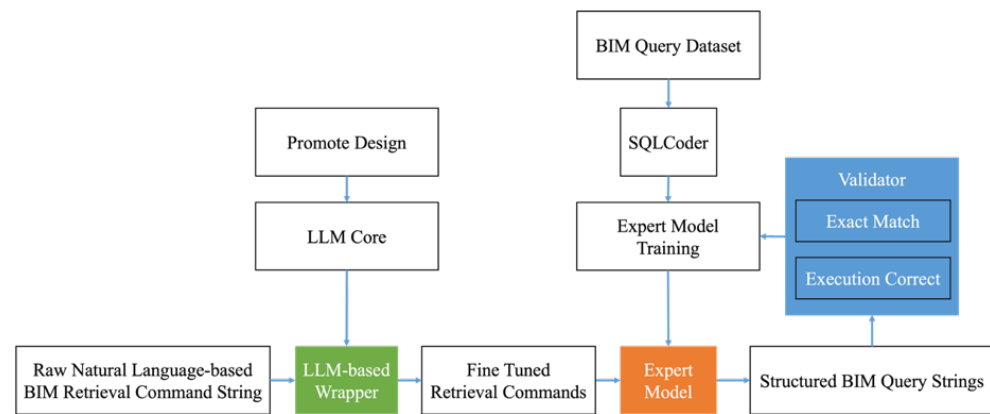


Figure 1. The framework of the BIMCoder Model.

3.2.1. SQLCoder Fine-Tuning-Based Expert Model

Fine-tuning SQLCoder, as the expert model, accepts natural language queries and generates structured BIM queries. It trained with Supervised Fine-Tuning (SFT) on the SQLCoder model. SQLCoder employs the Mistral model architecture, which excels at structured string generation.

The constructed dataset (presented in Section 3.1) was employed in the fine-tuning stage. Twelve types of precisely formulated and well-structured questions and their corresponding answers were used. These question categories were carefully designed to ensure specificity and accuracy. A total of 10% of the dataset was set aside for validation, while full-scale updates were employed for training. Given the small sample size of the dataset, the number of epochs was set to 10, with a learning rate of 0.000001 and a batch size of 2. The model learned and absorbed language patterns and query construction techniques from the structured dataset through fine-tuning training.

3.2.2. LLM-Based Wrapper

In real-world situations, applying the trained SQLCoder model directly to informal or vaguely worded questions often yields unsatisfactory results. Therefore, a pre-trained large language model is utilized to transform imprecise user queries into clearly formulated questions.

The ERNIE model was employed to convert raw queries into well-structured and explicitly directed queries. A specifically designed prompt was attached to guide the large language model in processing every raw query. Although this prompt is used with the ERNIE model, it does not mean ERNIE is the only choice. Any large language model that supports prompts can be used as a Wrapper model. Table 3 presents the prompt for the LLM-based Wrapper model.

Table 3. Prompt for LLM-based Wrapper Model.

This is a set of 12 precisely defined example queries. Please modify the given query to align with one of these types, ensuring structural consistency. If no appropriate match is found, retain the original query.

1. Provide a 3D view of the entire building.
2. Show only the “wall” components of the building.
3. Show both the “wall” and “door” components of the building.
4. Retrieve a specific component in the building using GUID: 16M94j0xL3f8bk1S7S.
5. Query components located within a spatial volume of $123 \times 123 \times 123$.
6. Retrieve properties of a specific component using GUID: 06gL_hkB14LhZ8gqCScQ.
7. Query the properties of a given building element.
8. List all component types and subtypes, including attributes such as OwnerHistory, Representation, and ObjectPlacement.
9. Show all walls with openings, including their subtypes, fills, and associated stairs.
10. Use GUID: 2udBPbKiCbZ8zbfpJmtDTM to locate a specific floor and list all elements on that floor.
11. Show only the external walls of the building.
12. List all IfcProduct components of type 57.2 along with their subtypes.

As shown in Table 4, once questions are transformed in this manner—more structured and directed—they can be passed on to the specialized BIMcoder model to generate more accurate query statements.

Table 4. Examples of LLM-based Wrapper Process Results for Raw Queries.

Original Question	Refined Question
Set up a $5 \times 5 \times 5$ dimension to search for things.	Locate and retrieve items within the building by setting the width, height, and depth dimensions to 5.
Find the floor with GUID 123 and show me what is inside.	Locate the floor with GUID 123 and display all elements within it.

3.2.3. Validator

In NL2SQL tasks, models are required to comprehend the semantics of natural language and generate SQL queries that are both syntactically valid and semantically accurate. To rigorously evaluate model performance, this study adopts two widely recognized evaluation metrics: Exact Match Accuracy (EM) and Execution Accuracy (EX), both of which are adapted to the context of structured query generation for Building Information Modeling (BIM).

Exact Match Accuracy measures the proportion of predictions that are exactly identical to the gold-standard SQL query. Formally, for each prediction, if the generated SQL query matches the reference SQL query token by token and structure by structure, the output is counted as correct. This metric places a strong emphasis on syntactic fidelity and full semantic coverage, and is particularly effective for detecting whether the model can completely and precisely encode all elements of a user's query, such as specific entity names, attribute constraints, and logical connectors.

Execution Accuracy, on the other hand, evaluates whether the result returned by executing the generated SQL query matches that of the reference query. Even if the predicted SQL differs in structure (e.g., uses different but equivalent expressions or JOIN orders), it is considered correct as long as the execution results are the same. This metric emphasizes functional correctness and reflects the model's ability to understand the user's intent and generate an executable query with equivalent semantics.

In the BIM domain, these metrics are calculated across 12 representative question categories, each corresponding to a specific application scenario—such as project cost estimation, component tracking, construction progress, spatial conflict detection, and compliance verification. For each category, we compute both EM and EX separately. The overall Exact Match Accuracy and Execution Accuracy are then derived by averaging across all question types, providing a holistic evaluation of model performance on BIM-oriented NL2SQL tasks.

To further enhance the reliability of structured query generation, we introduce a validator module that plays a critical role in ensuring both syntactic correctness and execution validity. The validator first checks whether the generated query conforms to the required IFCQL format using a built-in query format checker embedded in the BIM server platform. If the query fails to pass the syntax check or yields invalid execution results, the system provides immediate feedback to the user, indicating the cause of failure (e.g., unrecognized attributes, incorrect nesting, or unsupported filtering conditions). This feedback mechanism effectively mitigates hallucination issues and guides the user toward more accurate query reformulation, thus improving overall user experience and system robustness.

It is worth noting that certain challenges in the dataset may influence the robustness of these metrics. Specifically, queries involving GUID-based filtering or deeply nested IFC entities may introduce noise, as different syntactic representations may yield the same results (affecting EM) or produce execution discrepancies due to incomplete data linkage

(affecting EX). Additionally, since some fuzzy queries were derived from usage examples without expert annotation, there is potential for label bias or intent ambiguity.

To mitigate these factors, we performed basic data normalization and cleaning, including de-duplication, query canonicalization, and entity disambiguation. However, further improvements—such as integrating expert validation, filtering ambiguous queries, and implementing robust canonicalization pipelines—remain promising directions to enhance dataset quality and evaluation reliability in future work.

4. Experiment and Results

4.1. Benchmark LLMs' Performance on Structured Query Generation

The precise query test set consists of 240 samples, covering 12 common BIM query categories (Section 3.1, Table 1). The structure and phrasing of the queries in the precise query test set are consistent with those in the training set, and it is used to evaluate the translation performance of the three models.

4.1.1. Benchmark LLMs Selection

Three LLMs (ERNIE 3.0, Llama-13B, SQLCoder) are utilized as the benchmark of the experiment. ERNIE represents a large model optimized for Chinese, with a focus on knowledge-enhanced pre-training strategies, making it suitable for tasks such as structured knowledge extraction and instruction understanding in the BIM Chinese context. Llama-13B is a general-purpose, open-source English model that has not been specifically optimized for SQL or structured tasks, and is thus suitable for comparing the adaptability of general models in specialized domain tasks (baseline). SQLCoder represents the state-of-the-art fine-tuned model for structured query generation, having undergone reinforcement training for SQL-related tasks, and is specifically designed for the NL2SQL task. These models differ significantly in terms of architecture, pre-training objectives, and domain adaptability, making them typical representatives for a comprehensive comparative analysis.

The ERNIE 3.0, Llama-13B, and SQLCoder models have all fine-tuned on the collected dataset (Section 3.1), enabling them to convert user queries into SQL queries in JSON format for BIM. The unique architecture and knowledge-enhancement mechanism of ERNIE 3.0 require a larger batch size to stabilize the update of knowledge-related parameters, as well as a more significant number of epochs to ensure effective adaptation of the knowledge structure to domain-specific tasks. These factors highlight the need for substantial computational resources and meticulous tuning during the training process to achieve optimal performance in domain adaptation. The fine-tuning parameters for the three models are in Table 5.

Table 5. Fine-tuning Parameters for the Benchmark Models.

Parameters	ERNIE3.0	Llama-13B	SQLCoder
Epoch	15	5	5
Learning Rate	0.000003	0.000001	0.000001
Batch Size	16	1	1
Sequence Length	4096	4096	4096

Table 5 presents the key hyperparameter settings used to fine-tune the three benchmark models—ERNIE 3.0, LLaMA-13B, and SQLCoder—within the scope of this study. The final hyperparameter configurations were determined through a series of exploratory experiments, during which multiple combinations of training epochs and learning rates were tested to balance model convergence, training efficiency, and computational feasibility.

1. Epoch (Number of Training Epochs)

For ERNIE 3.0, training epoch values of 5, 10, and 15 were initially evaluated. The model demonstrated underfitting at lower epoch settings, while 15 epochs consistently yielded better convergence in domain adaptation scenarios, owing to its knowledge-enhanced architecture and deeper parameter structure. Thus, 15 epochs were adopted as the final setting to ensure sufficient model learning without overfitting. In contrast, for LLaMA-13B and SQLCoder, experiments with 3, 5, and 10 epochs showed that performance improvements plateaued after 5 epochs. Considering their more concise architectures and pre-training on code-related or structured language tasks, 5 epochs provided a good trade-off between accuracy and training time.

2. Learning Rate

Regarding the learning rate, a grid search was conducted over values ranging from 1×10^5 to 3×10^6 for ERNIE 3.0, and from 5×10^6 to 1×10^6 for LLaMA-13B and SQLCoder. The optimal learning rate for ERNIE 3.0 was determined to be 3×10^6 (0.000003), which ensured stable fine-tuning over its large-scale parameters while preventing gradient explosion. For LLaMA-13B and SQLCoder, a more conservative learning rate of 1×10^6 (0.000001) was found to yield better generalization and reduce the risk of overfitting on the relatively limited domain-specific training data.

3. Batch Size

The batch size selection was primarily constrained by hardware limitations. ERNIE 3.0 was trained with a batch size of 16, as it allowed for more stable gradient updates and better utilization of GPU memory. However, due to the significantly larger memory footprint of LLaMA-13B and SQLCoder, a batch size of 1 was used during fine-tuning. Although small batch sizes may introduce gradient variance, they can also enhance the model's ability to generalize and learn from detailed sample-level variations.

4. Sequence Length

A maximum sequence length of 4096 tokens was uniformly applied across all models. This choice was guided by the need to accommodate long and hierarchical query structures inherent to BIM-related tasks. Longer sequences help preserve the semantic consistency and structural integrity of complex instructions, which is critical for accurate structured query generation.

In summary, the final hyperparameter choices reflect a careful balance between empirical performance, model architecture, dataset scale, and computational constraints. The selection process was guided by empirical tuning and sensitivity testing, ensuring that each model was fine-tuned under conditions conducive to optimal performance on BIM-specific structured query tasks.

4.1.2. Exact Match Validation for Benchmark LLMs

The performance of ERNIE 3.0, LLaMA-13B, and SQLCoder models in accurately matching 12 types of building information query tasks (presented in Table 1) is illustrated in Table 6.

Based on the results presented in Table 6 and Figure 2, ERNIE 3.0 demonstrated the best overall performance, achieving a 100% accurate matching rate across 10 of the 12 query categories. However, its performance on query types T4 and T8 was less optimal, with matching rates of 50% and 78%, respectively.

The Llama-13B model exhibited unstable performance. While it achieved a 100% exact match rate in specific categories (such as T2, T4, and T12), its performance in other categories (including T3, T6, T7, T9, and T10) was significantly worse, with match rates of 0% or very low. The suboptimal performance of Llama-13B in this study is primarily due to its limited linguistic adaptability and domain adaptation capabilities. The model's original training corpus is predominantly in English, with insufficient pre-training in Chinese, leading to weak semantic understanding in Chinese NL2SQL tasks. This is particularly

evident in its failure to model Chinese contextual nuances and to map them to structured language effectively. Furthermore, unlike ERNIE 3.0, which incorporates knowledge-enhancement mechanisms, or SQLCoder, which is optimized for structured language, Llama-13B lacks pre-training or domain adaptation modules specifically designed for the Building Information Modeling (BIM) domain. As a result, it struggles to effectively handle the highly specialized semantic content and structural requirements in BIM queries.

Table 6. Exact Match Performance for Fine-tuned Benchmark LLMs.

Query Type	ERNIE3.0	Llama-13B	SQLCoder
T1	100%	58%	100%
T2	90%	100%	100%
T3	100%	10%	100%
T4	50%	100%	50%
T5	94%	74%	100%
T6	100%	0%	13%
T7	100%	0%	53%
T8	78%	64%	79%
T9	100%	0%	100%
T10	100%	0%	100%
T11	100%	88%	100%
T12	100%	100%	100%

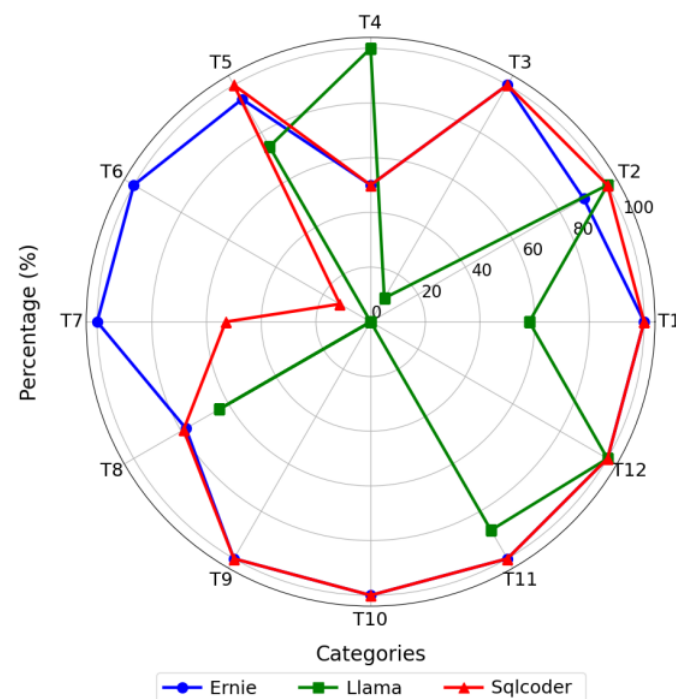


Figure 2. Comparison of Exact Matching Performance for Fine-tuned Benchmark LLMs.

SQLCoder performed comparably to ERNIE 3.0, achieving a 100% accurate matching rate in 10 categories. However, it exhibited significant weaknesses in problem types T6 and T7, with matching rates of only 13% and 53%, respectively.

4.1.3. Execution Accuracy of Benchmark LLMs

The Execution Accuracy reflects the effectiveness of the SQL query generated by the model, and the results are as follows.

Based on the results presented in Table 7 and Figure 3, Llama-13B exhibited the weakest performance among the evaluated models, primarily due to its reliance on an English-

centric training dataset, which limits its effectiveness in Chinese question-answering and SQL conversion tasks. In contrast, ERNIE 3.0 and SQLCoder each demonstrated distinct strengths.

Table 7. Execution Accuracy of Benchmark LLMs.

Query Type	ERNIE3.0	Llama-13B	SQLCoder
T1	100%	60%	100%
T2	90%	100%	100%
T3	100%	10%	100%
T4	50%	100%	50%
T5	95%	75%	100%
T6	100%	0%	15%
T7	100%	0%	55%
T8	80%	65%	80%
T9	100%	0%	100%
T10	100%	0%	100%
T11	100%	90%	100%
T12	100%	100%	100%

While ERNIE 3.0 underperformed the SQLCoder on problem types T2 and T5, it significantly outperformed the SQLCoder on problem types T6 and T7. This disparity highlights the two models' respective strengths. SQLCoder excels at generating structured language outputs, which aligns well with the more explicit task directions of T2 and T5. ERNIE 3.0 shows superior semantic understanding, which is crucial for addressing more ambiguous and complex problem types such as T6 (identifying specific GUID component properties) and T7 (distinguishing general component properties). Overall, the SQLCoder demonstrates a more substantial capability for structured language generation, whereas ERNIE exhibits more excellent proficiency in semantic comprehension.

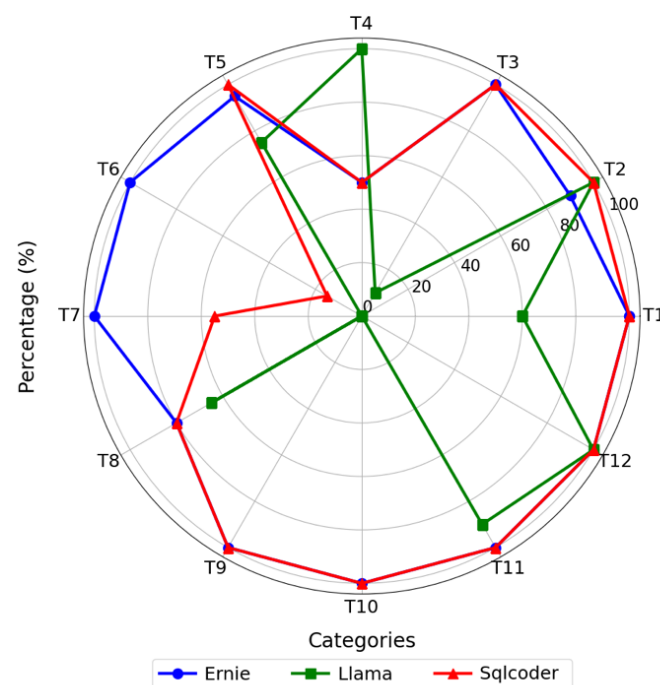


Figure 3. Comparison of Execution Accuracy Performance of Benchmark LLMs.

4.2. BIMCoder Validation

Fuzzy queries represent questions posed by non-expert users, making the test results for fuzzy queries more reflective of real-world scenarios. The fuzzy query test set consists

of 240 samples, divided into 12 categories, like the precise query test set, but with more ambiguous phrasing. Additionally, the fuzzy query test set provides an opportunity to formally compare the performance of BIMcoder with the other three baseline models.

4.2.1. Exact Match Validation on Fuzzy Query Dataset

Based on the results presented in Table 8 and Figure 4, the performance of each model was further evaluated on colloquial and fuzzy queries, where the characteristics of the queries tend to be more informal and complex.

Table 8. Exact Match Performance on Fuzzy Queries.

Query Type	BIMCoder	ERNIE3.0	Llama-13B	SQLCoder
T1	92%	100%	46%	65%
T2	90%	95%	100%	50%
T3	79%	95%	0%	60%
T4	91%	63%	100%	45%
T5	85%	44%	38%	10%
T6	100%	95%	10%	15%
T7	81%	90%	0%	0%
T8	100%	85%	0%	0%
T9	94%	70%	28%	85%
T10	53%	28%	0%	0%
T11	99%	100%	20%	100%
T12	82%	40%	30%	65%

ERNIE 3.0 maintained strong performance, outperforming other models in most cases. It demonstrated significant advantages in categories such as T6, T7, T8, T9, and T11. Even when confronted with the most complex fuzzy query, ERNIE consistently achieved high precision in matching and Execution Accuracy.

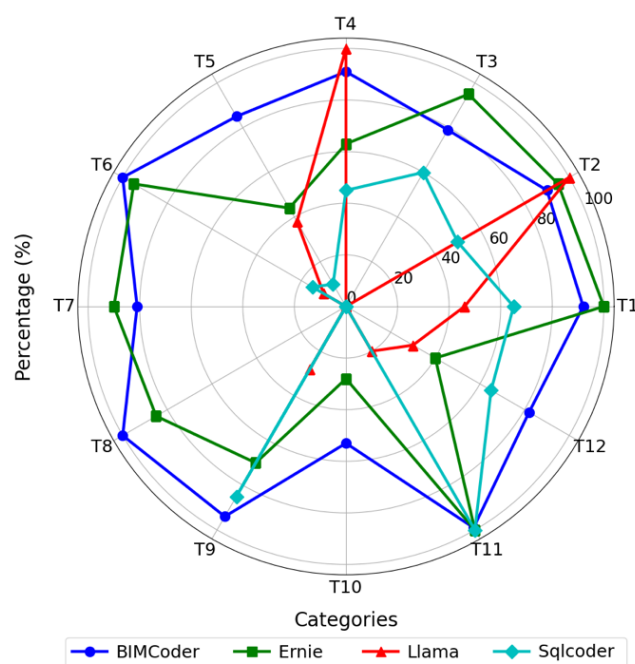


Figure 4. Comparison of Exact Matching Performance on Fuzzy Queries.

The performance of the SQLCoder on fuzzy queries was less stable than its performance on precise queries. Notably, its precision matching rate and Execution Accuracy decreased significantly in categories such as T5, T7, and T10.

Llama continued to exhibit substantial variability in performance on fuzzy queries. Many categories showed very low accurate matching, underscoring the model's limitations in handling informal or complex natural language queries.

The proposed BIMCoder model, which combines ERNIE 3.0's semantic understanding capabilities with SQLCoder's structural language generation strengths, demonstrated exceptional performance on fuzzy queries. It achieved higher accurate matching than any individual model across almost all categories. In particular, the fusion model significantly improved success rates in categories such as T5, T7, T8, T9.

4.2.2. Execution Accuracy on Fuzzy Queries

Based on the results presented in Table 9 and Figure 5, when addressing fuzzy problems, the test results indicate that the ERNIE 3.0 model significantly outperforms Llama and SQLCoder. The challenges posed by colloquial and diverse query methods in the fuzzy dataset require strong semantic understanding, an area where ERNIE excels. The fusion model (BIMCoder) further enhances performance by leveraging ERNIE 3.0's understanding ability and SQLCoder's structural language generation capability. This integration leads to more stable and accurate results in exact matching and Execution Accuracy, surpassing even the ERNIE fine-tuned model.

Table 9. Exact Match Performance on Fuzzy Queries.

Query Type	BIMCoder	ERNIE3.0	Llama-13B	SQLCoder
T1	95%	100%	50%	65%
T2	90%	95%	100%	50%
T3	80%	95%	0%	60%
T4	95%	63%	100%	45%
T5	85%	45%	40%	10%
T6	100%	95%	10%	15%
T7	85%	90%	0%	0%
T8	100%	85%	0%	0%
T9	95%	70%	30%	85%
T10	55%	30%	0%	0%
T11	100%	100%	20%	100%
T12	85%	40%	30%	65%

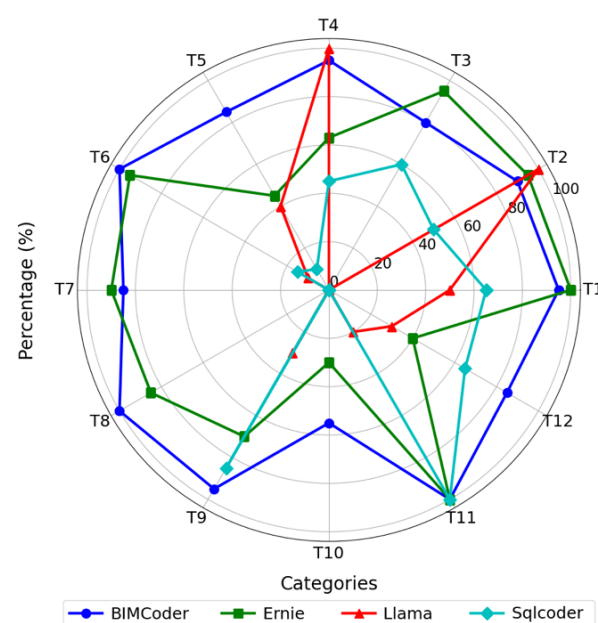


Figure 5. Comparison of Execution Accuracy on Fuzzy Queries.

4.2.3. Analysis of BIMCoder Model Test Results

In the above experiment, the BIMCoder model achieved the highest overall performance. This can be attributed primarily to the generalization capabilities of the ERNIE 3.0 model. With the assistance of the LLM-based Wrapper model, the expert model can concentrate on structured query generation by receiving cleaner, normalized inputs.

However, Exact Match Accuracy significantly declined when the model encountered fuzzified or colloquially expressed query sets. To mitigate this issue, we adopted a two-stage strategy: first, leveraging the ERNIE 3.0 model's robust generalization ability, the system identifies key entities and semantic structures from vague user queries and converts them into more standardized representations; then, these normalized queries are passed to the fine-tuned SQLCoder model to generate precise and executable BIM queries.

The combination of the ERNIE 3.0 and SQLCoder models forms a collaborative architecture that capitalizes on ERNIE 3.0's strengths in interpreting complex or ambiguous language and SQLCoder's specialization in generating accurate SQL syntax. This collaborative mechanism proves effective in handling a wide variety of BIM-related natural language queries, especially in scenarios where language variability and user intent ambiguity are prominent. It demonstrates that the proposed pipeline can expand the applicability of NL2SQL techniques to real-world data analysis and BIM system integration.

Despite these strengths, several limitations were observed in edge cases. For instance, in task T10, BIMCoder failed to generate a correct structured query. Analysis reveals that the failure is due to the model misinterpreting the relationship between the GUID and the semantic role of "floor." Specifically, the fuzzy phrasing caused confusion between spatial containers (e.g., floors, zones, buildings) and component elements, resulting in a query structure that either neglected the containment hierarchy or retrieved unrelated elements. This example highlights a known limitation of the current pipeline: when queries rely on implicit IFC hierarchy knowledge or reference ambiguous spatial entities using GUIDs, the model may misrepresent entity roles or generate logically inconsistent queries.

This failure case underscores the current limitations of the BIMCoder pipeline in accurately interpreting spatial semantics and IFC containment hierarchies. To address these issues, subsequent work should emphasize the incorporation of hierarchical reasoning mechanisms, the expansion of training data to include more hierarchy-aware examples, and the integration of domain-specific semantic validation modules. Such enhancements are anticipated to improve the model's interpretability and reliability in complex, real-world BIM environments.

5. Discussion

5.1. Generalization Ability

5.1.1. Native Models

The SQLCoder model excels in handling direct queries, specific attribute queries, and complex spatial-related query tasks, achieving nearly perfect exact and Execution Accuracy. These results are attributed to its design, specifically tailored for natural-language-to-SQL conversion, and the incorporation of a grouped query attention mechanism and intermediate padding techniques, enhancing the model's ability to manage complex SQL queries.

Although ERNIE's precision in certain tasks, such as querying specific shapes of components or identifying wall openings, falls short compared to the other two models, it exhibits strong generalization capabilities when dealing with ambiguous and colloquial natural language queries. This is due to ERNIE's superior ability to comprehend and restructure natural language inputs.

After fine-tuning in a Chinese context, the Llama model excels in specific tasks, such as identifying individual components within a building and querying wall openings and their

connections to other components. However, its accuracy declined when handling queries involving multiple attributes or specific conditions. This indicates that Llama adapts well to the Chinese language environment after appropriate fine-tuning, but its ability to handle complex attribute queries is somewhat limited.

5.1.2. BIMCoder Model

Using the ERNIE model to preprocess queries and extract essential information, followed by SQLCoder to generate the SQL statements, significantly improves the Exact Match Accuracy to 90%. This demonstrates that ERNIE's strength in understanding and reconstructing natural language queries and SQLCoder's proficiency in SQL generation complement each other, thereby enhancing the overall generalization capability of the system.

ERNIE's strong generalization capability arises from its ability to understand the context and intent of user input. In contrast, while SQLCoder and Llama perform better on specific tasks, their generalization capabilities may be limited by the limited scope of their training datasets.

5.2. Hallucination Problems

The hallucination problem typically refers to instances where a model generates outputs that are inconsistent with the input or context. In the context of natural language to structured query tasks, such as NL2SQL, this phenomenon may manifest as the generated query not aligning with the intent of the original natural language query. In the case of BIM information retrieval, the hallucination problem could lead to incorrect or imprecise queries being generated, affecting the accuracy and reliability of BIM software interactions.

5.2.1. Native Models

1. SQLCoder Model

The SQLCoder model is specifically designed for converting natural language into SQL queries. Due to its specialized architecture, it is optimized for handling structured query generation, which results in relatively fewer hallucination issues. SQLCoder has demonstrated an ability to accurately translate straightforward queries into SQL, typically reflecting the intent of the input query with high precision. This makes SQLCoder less prone to hallucinations compared to more general-purpose models. However, it is not entirely free from limitations. In cases where the input query is highly ambiguous or involves complex attributes not commonly seen in the training data, SQLCoder may still exhibit minor hallucination problems, where the generated query does not fully align with the original intent.

2. ERNIE Model

The ERNIE model, with its strong generalization capabilities, excels at processing a wide range of natural language queries, including ambiguous and colloquial expressions. This ability to generalize is beneficial when dealing with natural language queries that are not strictly formal or structured. However, this generalization power can sometimes backfire, leading to the generation of SQL queries that do not fully capture the intent of the original query. This issue is particularly noticeable in cases where the input query includes subtle nuances, domain-specific terminology, or indirect references that the ERNIE model might misinterpret. In such instances, ERNIE may produce a query that appears plausible but deviates from the expected result, leading to hallucination issues.

3. Llama Model

The Llama model, while powerful, shows some limitations when handling complex queries involving multiple attributes or intricate conditions. This becomes especially

apparent in tasks where the natural language query includes various parameters or requires the model to make inferences based on multiple attributes. As a result, Llama may generate SQL queries that do not align with the user's intent, leading to mismatched outputs. The model's lower accuracy in handling such complex queries suggests that it may struggle to generate queries that accurately reflect the specifics of the natural language input, resulting in hallucination problems. These issues are particularly problematic in scenarios where precision is crucial, such as BIM information retrieval, where small deviations can lead to significant discrepancies in the final result.

5.2.2. BIMCoder Model

While the individual ERNIE and Llama models exhibit some hallucination issues, the BIMCoder model, which combines ERNIE for query preprocessing and key information extraction with SQLCoder for query generation, is designed to mitigate these challenges. The fusion of these two models leverages their respective strengths to reduce the occurrence of hallucination problems, particularly in the context of BIM information retrieval.

In this model, ERNIE's capabilities in understanding and extracting key information from natural language queries allow it to preprocess and disambiguate complex input. This step ensures that the model has a clearer understanding of the query's intent before SQLCoder is tasked with generating the corresponding SQL query. SQLCoder, with its high exact matching rate and specialized design for structured query generation, helps correct any potential deviations introduced by ERNIE. This combination makes BIMCoder more robust in handling a wide variety of queries while reducing the risk of hallucinations.

Despite these advancements, hallucination issues can still arise in certain situations. For example, when a query contains highly specialized domain-specific terminology or when the training data does not cover a broad enough range of query types, the model might generate a SQL query that misrepresents the user's intent. This could happen due to limitations in the training data, which may not fully capture the diversity of possible query forms, or due to inherent structural limitations within the models themselves.

However, by leveraging the complementary strengths of ERNIE and SQLCoder, the BIMCoder model is better equipped to handle ambiguities and edge cases. ERNIE's preprocessing can filter out irrelevant or noisy information, while SQLCoder's precision in query generation ensures that the final output closely matches the original intent. The combination of these models significantly reduces the likelihood of hallucination, making BIMCoder a more reliable tool for natural language-based BIM information retrieval.

Overall, while hallucination remains a challenge in natural language processing tasks, the BIMCoder model offers a promising solution by combining the best of both ERNIE and SQLCoder. Future work could focus on further enhancing the model's robustness to hallucinations by expanding training datasets, refining the fusion process, and incorporating additional strategies to mitigate errors in query generation.

6. Conclusions

In this study, we not only explored and compared the performance of three pre-trained language models—ERNIE, Llama, and SQLCoder—in executing NL2SQL tasks within Building Information Modeling (BIM) scenarios but also constructed a high-quality dataset tailored explicitly for the BIM field to support this advanced research. Through customized model fine-tuning, we found that the SQLCoder model excels in most standard query tasks. In contrast, the ERNIE model stands out for its robust generalization ability in handling fuzzy and colloquial natural language queries. By combining the strengths of these two models, we designed an innovative two-stage query scheme: first, the ERNIE model is

used for raw language input, and then, a fine-tuned SQLCoder model is used to generate precise, structured BIM query instructions, significantly improving query efficiency.

However, our research also revealed some potential limitations. The accuracy of the models when dealing with queries involving specific complex conditions, such as retrieving components with specific geometric shapes, still has room for improvement. Additionally, due to specific application scenarios in the BIM field, such as disaster planning, actual cases are relatively rare, resulting in a lack of relevant training data, affecting the model's performance in these scenarios. Furthermore, optimizing model parameters requires dynamic adjustments based on different corpora, posing challenges for the continuous optimization and maintenance of the model.

Future research will focus on exploring multiple directions. On the one hand, we will utilize data augmentation techniques and transfer learning strategies to enhance the performance of models in data-scarce environments. On the other hand, developing an automated parameter search mechanism to achieve adaptive optimization of the model configuration enhances its cross-scenario adaptability and practicality. More importantly, we will strive to integrate the advantages of different models, such as ERNIE's generalization ability and BIMcoder's precise matching ability, to create a more intelligent and flexible query system. Meanwhile, with the continuous development of BIM technology and the increasingly diverse application scenarios, continuous data accumulation and model iteration will be vital to improving the system's long-term performance.

In summary, this study provides an innovative solution for natural language query processing in the BIM field and constructs a specialized dataset to promote research progress in this area. It also clarifies the direction and focus of future research, laying a solid foundation for promoting the continuous innovation and application of BIM technology.

Author Contributions: Conceptualization, B.L. and H.C.; methodology, B.L. and H.C.; software, B.L. and H.C.; validation, B.L. and H.C.; writing—original draft preparation, B.L.; writing—review and editing, H.C.; funding acquisition and resources, H.C.; supervision, H.C. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Natural Science Foundation of China (72301250).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in the study are included in the article; further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Lin, J.; Hu, Z.; Zhang, J.; Yu, F. A Natural-Language-Based Approach to Intelligent Data Retrieval and Representation for Cloud BIM. *Comput. Aided Civ. Infrastruct. Eng.* **2015**, *31*, 18–33. [\[CrossRef\]](#)
2. Wang, J.; Gao, X.; Zhou, X.; Xie, Q. Multi-scale Information Retrieval for BIM using Hierarchical Structure Modelling and Natural Language Processing. *J. Inf. Technol. Constr.* **2021**, *26*, 409–426. [\[CrossRef\]](#)
3. Yin, M.; Li, H.; Wu, Z.; Tang, L. Information Requirement Analysis for Establishing BIM-Oriented Natural Language Interfaces. In *International Conference on Sustainable Buildings and Structures Towards a Carbon Neutral Future*; Springer: Singapore, 2023.
4. Revuelta-Martínez, A.; Rodríguez, L.; García-Varea, I.; Montero, F. Multimodal interaction for information retrieval using natural language. *Comput. Stand. Interfaces* **2013**, *35*, 428–441. [\[CrossRef\]](#)
5. Zhao, Y.; Taib, N. Cloud-based building information modelling (Cloud-BIM): Systematic literature review and bibliometric-qualitative analysis. *Autom. Constr.* **2022**, *142*, 104468. [\[CrossRef\]](#)
6. Zheng, J.; Fischer, M. Dynamic prompt-based virtual assistant framework for BIM information search. *Autom. Constr.* **2023**, *155*, 105067. [\[CrossRef\]](#)

7. Elghaish, F.; Chauhan, J.K.; Matarneh, S.; Rahimian, F.P.; Hosseini, M.R. Artificial intelligence-based voice assistant for BIM data management. *Autom. Constr.* **2022**, *140*, 104320. [CrossRef]
8. Huang, C. Application of BIM Technology Based on Autodesk Revit in Construction and Installation Engineering. In Proceedings of the 2017 3rd International Conference on Economics, Social Science, Arts, Education and Management Engineering (ESSAEME 2017), Huhhot, China, 29–30 July 2017.
9. Aish, R. Bentley Systems. Migration from an Individual to an Enterprise Computing Model and Its Implications for AEC Research. 2020. Available online: <https://www.bentley.com/software/building-design/> (accessed on 30 April 2025).
10. Bernard, F. Dassault systèmes: A CAD success story. *IEEE Ann. Hist. Comput.* **2024**, *46*, 124–135. [CrossRef]
11. Liu, Z.; Wang, Y. Real-Time Interactive Online 3D Graphical User Interface (GUI) Technical Implementation and Usability Test for Architectural Technical Teaching. In *Lecture Notes in Computer Science*; Springer: Berlin/Heidelberg, Germany, 2020; pp. 452–469. [CrossRef]
12. Sampaio, A.Z. 4D/BIM model linked to VR technology. In Proceedings of the Virtual Reality International Conference-Laval Virtual 2017, Laval, France, 22–24 March 2017; pp. 1–4.
13. Loporcaro, G.; Bellamy, L.; McKenzie, P.; Riley, H. Evaluation of Microsoft HoloLens Augmented Reality Technology as a construction checking tool. *Constr. Innov. Int. J.* **2019**, 124–135.
14. De Sa, J.L.P.; Alfaro, P. Implementing common data environments in architectural technology studies. *Build. Inf. Model. Des. Constr. Oper. IV* **2021**, *205*, 67.
15. Shin, S.; Issa, R.R. BIMASR: Framework for voice-based BIM information retrieval. *J. Constr. Eng. Manag.* **2021**, *147*, 04021124. [CrossRef]
16. Linares-Garcia, D.A.; Roofigari-Esfahan, N. Practical Deployment of BIM-Interoperable Voice-Based Intelligent Virtual Agent to Support Construction Worker Productivity. *J. Constr. Eng. Manag.* **2024**, *150*, 04024153. [CrossRef]
17. Zhoui, H.; Wong, M.O.; Ying, H.; Lee, S. A framework of a multi-user voice-driven BIM-based navigation system for fire emergency response. In Proceedings of the 26th International Workshop on Intelligent Computing in Engineering (EG-ICE 2019), The European Group for Intelligent Computing in Engineering (EG-ICE), Leuven, Belgium, 30 June–3 July 2019.
18. Park, D.; Kim, E. Method of interacting between humans and conversational voice agent systems. *Heliyon* **2024**, *10*. [CrossRef] [PubMed]
19. Bello, S.A.; Oyedele, L.O.; Akanbi, L.A.; Bello, A.L. Cloud computing for chatbot in the construction industry: An implementation framework for conversational-BIM voice assistant. *Digit. Eng.* **2025**, *5*, 100031. [CrossRef]
20. Hu, Z.; Zhang, J. BIM oriented intelligent data mining and representation. In Proceedings of the 30th CIB W78 International Conference, Beijing, China, 9–12 October 2013; pp. 280–289.
21. Nabavi, A.; Ramaji, I.; Sadeghi, N.; Anderson, A. Leveraging Natural Language Processing for Automated Information Inquiry from Building Information Models. *J. Inf. Technol. Constr.* **2023**, *28*, 266–285. [CrossRef]
22. Xie, Q.; Zhou, X.; Wang, J.; Gao, X.; Chen, X.; Liu, C. Matching real-world facilities to building information modeling data using natural language processing. *IEEE Access* **2019**, *7*, 119465–119475. [CrossRef]
23. Wu, S.; Shen, Q.; Deng, Y.; Cheng, J. Natural-language-based intelligent retrieval engine for BIM object database. *Comput. Ind.* **2019**, *108*, 73–88. [CrossRef]
24. Yin, M.; Tang, L.; Webster, C.; Xu, S.; Li, X.; Ying, H. An ontology-aided, natural language-based approach for multi-constraint BIM model querying. *J. Build. Eng.* **2023**, *76*, 107066. [CrossRef]
25. Gao, G.; Liu, Y.S.; Wang, M.; Gu, M.; Yong, J.H. A query expansion method for retrieving online BIM resources based on Industry Foundation Classes. *Autom. Constr.* **2015**, *56*, 14–25. [CrossRef]
26. Ding, L.; Zhong, B.; Wu, S.; Luo, H. Construction risk knowledge management in BIM using ontology and semantic web technology. *Saf. Sci.* **2016**, *87*, 202–213. [CrossRef]
27. Li, X.; Yang, D.; Yuan, J.; Donkers, A.; Liu, X. BIM-enabled semantic web for automated safety checks in subway construction. *Autom. Constr.* **2022**, *141*, 104454. [CrossRef]
28. Zhou, Y.; Bao, T.; Shu, X.; Li, Y.; Li, Y. BIM and ontology-based knowledge management for dam safety monitoring. *Autom. Constr.* **2023**, *145*, 104649. [CrossRef]
29. Guo, D.; Onstein, E.; Rosa, A.D.L. An approach of automatic SPARQL generation for BIM data extraction. *Appl. Sci.* **2020**, *10*, 8794. [CrossRef]
30. Cursi, S.; Simeone, D.; Coraglia, U. An ontology-based platform for BIM semantic enrichment. In Proceedings of the ShoCK—Sharing of Computable Knowledge, International Conference on Education and Research in Computer Aided Architectural Design in Europe, Rome, Italy, 20–22 September 2017; Volume 2, pp. 649–656.
31. Fahad, M.; Bus, N.; Fies, B. Semantic BIM reasoner for the verification of IFC Models. In *eWork and eBusiness in Architecture, Engineering and Construction*; CRC Press: Boca Raton, FL, USA, 2018; pp. 361–368.
32. Solihin, W.; Eastman, C.; Lee, Y.C.; Yang, D.H. A simplified relational database schema for transformation of BIM data into a query-efficient and spatially enabled database. *Autom. Constr.* **2017**, *84*, 367–383. [CrossRef]

33. Alzraiee, H. Cost estimate system using structured query language in BIM. *Int. J. Constr. Manag.* **2022**, *22*, 2731–2743. [[CrossRef](#)]
34. Demirdöğen, G.; Işık, Z.; Arayıcı, Y. BIM-based big data analytic system for healthcare facility management. *J. Build. Eng.* **2023**, *64*, 105713. [[CrossRef](#)]
35. Li, B.; Luo, Y.; Chai, C.; Li, G.; Tang, N. The dawn of natural language to SQL: Are we fully ready? *arXiv* **2024**, arXiv:2406.01265. [[CrossRef](#)]
36. Sun, Y.; Wang, S.; Feng, S.; Ding, S.; Pang, C.; Shang, J.; Liu, J.; Chen, X.; Zhao, Y.; Lu, Y.; et al. Ernie 3.0: Large-scale knowledge enhanced pre-training for language understanding and generation. *arXiv* **2021**, arXiv:2107.02137.
37. Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv* **2023**, arXiv:2307.09288.
38. Thakkar, H.; Manimaran, A. Comprehensive Examination of Instruction-Based Language Models: A Comparative Analysis of Mistral-7B and Llama-2-7B. In Proceedings of the 2023 International Conference on Emerging Research in Computational Science (ICERCS), Coimbatore, India, 7–9 December 2023; pp. 1–6.
39. Espejel, J.L.; Ettifouri, E.H.; Alassan, M.S.Y.; Chouham, E.M.; Dahhane, W. GPT-3.5, GPT-4, or BARD? Evaluating LLMs reasoning ability in zero-shot setting and performance boosting through prompts. *Nat. Lang. Process. J.* **2023**, *5*, 100032. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.